



**UNIVERSIDAD TECNOLÓGICA METROPOLITANA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INFORMÁTICA Y COMPU-
TACIÓN
ESCUELA DE INFORMÁTICA**

SISTEMA DE CONTROL DE ACCESO VEHICULAR AUTOMATIZADO MEDIANTE RECONOCIMIENTO DE PATENTES PARA LA UNIVERSIDAD TECNOLÓGICA METROPOLITANA, SEDE CAMPUS ÑUÑO A

**TRABAJO DE TITULACIÓN PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN MENCION INFORMÁTICA**

AUTORES:

**MOLINA ZUÑIGA, EZEQUIEL JEREMÍAS
VALDÉS VALDÉS, BENJAMÍN IGNACIO**

PROFESOR GUÍA:

CRISTI CAPSTICK, MICHAEL EMILIO

**SANTIAGO – CHILE
2026**

AUTORIZACIÓN PARA LA REPRODUCCIÓN DEL TRABAJO DE TITULACIÓN

1. Identificación del Trabajo de Titulación

Nombre del(os) alumno(s):

RUT:

Dirección:

E-mail:

Teléfono:

Título de la tesis:

Escuela:

Carrera o programa:

Título al que opta:

2. Autorización de Reproducción (seleccione una opción)

a) Este trabajo de titulación no puede reproducirse o transmitirse bajo ninguna forma o por ningún medio o procedimiento, sin permiso escrito del(os) autor(es), exceptuando la cita bibliográfica, resumen y metadatos que acreditan al trabajo y a su(s) autor(es).

Fecha: _____

Firma: _____

b) Se autoriza la reproducción total o parcial de este trabajo de titulación, con fines académicos, por cualquier medio o procedimiento, incluyendo la cita bibliográfica que acredita al trabajo y a su autor.

En consideración a lo anterior, se autoriza su reproducción de forma (marque con una X):

☐ Inmediata

☐ A partir de la siguiente fecha: _____ (mes/año)

Fecha: _____

Firma: _____

Esta autorización se otorga en el marco de la ley N°17.336 sobre Propiedad Intelectual, con carácter gratuito y no exclusivo para la Institución.

NOTA OBTENIDA:

Firma y timbre de la autoridad
responsable

A mi familia, por su apoyo incondicional...

AGRADECIMIENTOS

Agradezco a...

TABLA DE CONTENIDO

INTRODUCCIÓN	1
1. Marco Teórico	2
1.1. Fundamentos y Arquitectura de los Sistemas ALPR	2
1.1.1. Definición y Evolución: De la Visión Clásica al <i>Deep Learning</i>	3
1.1.2. Los Pilares del Sistema: Detección (LPD) y Reconocimiento (LPR)	4
1.1.3. Flujos de Procesamiento: <i>Pipelines</i> Secuenciales vs. Sistemas Integrados (<i>End-to-End</i>)	5
1.2. Análisis del Estado del Arte en Detección (LPD)	6
1.2.1. La Evolución de la Familia YOLO (<i>You Only Look Once</i>) . .	6
1.2.2. Arquitecturas de Detección Alternativas	8
1.3. Motores de Extracción de Texto (LPR) y OCR	9
1.3.1. Metodologías de Reconocimiento	9
1.3.2. Benchmarking de Motores y Viabilidad en el Borde	10
1.4. Hardware de Borde y Optimización (<i>Edge AI</i>)	12
1.4.1. Ecosistemas de Cómputo en el Borde: Arquitecturas y Rendimiento	12
1.4.2. Técnicas de Compilación y Cuantización	13
1.5. Robustez en Entornos No Restringidos	15
1.5.1. Técnicas de Preprocesamiento Crítico	15
1.5.2. Módulos de Mejora Dinámica	15
1.5.3. Datos Sintéticos y Aumento (<i>Data Augmentation</i>)	15
1.6. Consistencia Temporal y Seguimiento (<i>Tracking</i>)	15

1.6.1.	Algoritmos de <i>Tracking</i>	15
1.6.2.	Votación Temporal e Interpolación	15
1.7.	Contexto Normativo y Operacional Chileno	15
1.7.1.	Estándares de Patentes en Chile	15
1.7.2.	Requerimientos Funcionales del Prototipo	15
2.	Metodología	16
2.1.	Diseño metodológico	16
2.2.	Fases o etapas del proyecto	16
2.2.1.	Etapa 1: Levantamiento de requerimientos y análisis	16
2.2.2.	Etapa 2: Diseño y arquitectura	16
2.2.3.	Etapa 3: Desarrollo e implementación	16
2.2.4.	Etapa 4: Pruebas y validación	17
2.3.	Herramientas y tecnologías	17
2.4.	Sujetos o elementos de estudio (Población y Muestra)	17
3.	Desarrollo de la Solución	18
3.1.	Arquitectura de la solución	18
3.2.	Modelo de datos y almacenamiento	18
3.3.	Implementación detallada de componentes	18
3.3.1.	Componente o módulo backend	18
3.3.2.	Componente o módulo frontend (Interfaz de usuario)	18
3.4.	Casos de uso principales y flujos de trabajo	19
4.	Resultados y Discusión	20
4.1.	Plan de pruebas y validación	20
4.1.1.	Pruebas unitarias y de integración	20

4.1.2. Pruebas de rendimiento o estrés	20
4.1.3. Pruebas de aceptación de usuarios	20
4.2. Presentación de resultados	20
4.3. Discusión de los resultados	21
CONCLUSIONES	22
BIBLIOGRAFÍA	23
GLOSARIO	26
ANEXOS	27
A. Manual de Instalación y Configuración	28
A.1. Requisitos del sistema	28
A.2. Instrucciones de instalación y despliegue	28
B. Código Fuente y Configuración Relevante	29

ÍNDICE DE TABLAS

4.1. Ejemplo de tabla de resultados	21
---	----

ÍNDICE DE ILUSTRACIONES

RESUMEN

[Escribe aquí tu resumen...]

PALABRAS CLAVES:

Palabra1, Palabra2, Palabra3, Palabra4, Palabra5.

ABSTRACT

[Write your abstract here...]

KEYWORDS:

Keyword1, Keyword2, Keyword3, Keyword4, Keyword5.

INTRODUCCIÓN

[Escribe aquí el contexto y la motivación de tu trabajo...]

CAPÍTULO 1

MARCO TEÓRICO

1.1 Fundamentos y Arquitectura de los Sistemas ALPR

El Reconocimiento Automático de Placas Patentes (ALPR, por sus siglas en inglés) constituye una tecnología de visión computacional crítica dentro de las infraestructuras de Sistemas de Transporte Inteligente (ITS), permitiendo la identificación de vehículos en tiempo real mediante la extracción de caracteres alfanuméricos a partir de imágenes o secuencias de video (Abdul-Sattar & Al Azzo, 2026; Kuyunsa et al., 2025; Laroca et al., 2025). Su implementación es un pilar fundamental en aplicaciones de seguridad pública, gestión automatizada de estacionamientos, control de acceso a áreas restringidas y aplicación de leyes de tránsito (Buleu et al., 2025; Xu et al., 2026; Zherzdev & Gruzdev, 2018). La evolución de estos sistemas responde a la necesidad de superar las ineficiencias de los modelos de vigilancia manual, los cuales resultan lentos, costosos y altamente propensos a errores humanos, limitando la escalabilidad operativa en entornos urbanos densos (Meesad & Thumthong, 2025; Wijaya & Soewito, 2024; Xu et al., 2026).

La eficacia de un sistema ALPR robusto se mide por su capacidad de operar en entornos no restringidos (*unconstrained environments*), donde debe procesar datos bajo condiciones ambientales adversas y estocásticas (Abdul-Sattar & Al Azzo, 2026; Vargoorani & Suen, 2024; Xu et al., 2026). Entre los principales desafíos técnicos se encuentran las variaciones extremas de iluminación —como el deslumbramiento solar o la penumbra nocturna—, las obstrucciones por suciedad o agentes climáticos (lluvia, niebla, nieve) y las distorsiones geométricas causadas por el movimiento del vehículo o el ángulo de captura de la cámara (Buleu et al., 2025; Mobeen, 2026; Sonnara et al., 2025; Xiong et al., 2026). Por ello, la viabilidad de un prototipo operativo depende del cumplimiento de un presupuesto de latencia estricto, habitualmente situado en torno a los 100 ms por cuadro, asegurando una toma de decisiones inmediata que es indispensable para el flujo vehicular en tiempo real (Kuyunsa et al., 2025; Sonnara et al., 2025).

1.1.1 Definición y Evolución: De la Visión Clásica al *Deep Learning*

La arquitectura de los sistemas ALPR ha experimentado una transformación paradigmática, migrando desde métodos deterministas basados en el procesamiento de imágenes tradicional hacia redes neuronales altamente parametrizadas (Mobeen, 2026; Xu et al., 2026). Inicialmente, la localización y el reconocimiento dependían de descriptores manuales (*handcrafted features*) y heurísticas de bajo nivel, tales como el detector de bordes de Canny, la segmentación por color (espacios HSV/YUV) y operaciones morfológicas de dilatación para aislar candidatos a placas (Abdul-Sattar & Al Azzo, 2026; Mobeen, 2026; Xu et al., 2026). En estas etapas tempranas, técnicas como la Transformada de Radon eran fundamentales para la corrección de inclinación (*tilt correction*), mientras que la clasificación de caracteres se realizaba mediante algoritmos de coincidencia de plantillas (*template matching*) o máquinas de vectores de soporte (SVM) (Buleu et al., 2025; Sonnara et al., 2025).

A pesar de su eficiencia computacional inicial, estos métodos clásicos presentan una fragilidad crítica ante la variabilidad del entorno real, degradando drásticamente su precisión frente al desenfoque por movimiento, oclusiones parciales o cambios bruscos en la iluminación ambiental (Meesad & Thumthong, 2025; Sonnara et al., 2025; Vargoorani & Suen, 2024). Esta limitación motivó la adopción del Aprendizaje Profundo (*Deep Learning*), específicamente mediante Redes Neuronales Convolucionales (CNN), las cuales permiten la extracción automatizada de características jerárquicas abstractas directamente desde los datos de píxeles *raw*, eliminando la necesidad de ingeniería de características manual (Abdul-Sattar & Al Azzo, 2026; Swati et al., 2024; Xu et al., 2026).

La evolución dentro del aprendizaje profundo se divide en dos enfoques de detección: los detectores de dos etapas (ej. Faster R-CNN) y los de una sola etapa (familia YOLO) (Abdul-Sattar & Al Azzo, 2026; Swati et al., 2024). Mientras que los modelos de dos etapas priorizan la precisión mediante una Red de Propuesta de Regiones (RPN), los modelos de una sola etapa redefinieron la detección como un problema de regresión unificado, logrando un balance superior entre latencia y exactitud, lo que ha consolidado a arquitecturas como YOLOv8, YOLOv10 y YOLOv12 como el estándar para aplicaciones de vigilancia y con-

trol de acceso en tiempo real (Buleu et al., 2025; Mobeen, 2026; Sonnara et al., 2025).

1.1.2 Los Pilares del Sistema: Detección (LPD) y Reconocimiento (LPR)

Un sistema ALPR de alto desempeño se fundamenta en la integración de dos procesos computacionales secuenciales y estrechamente acoplados: la Detección de Placas Patentes (LPD, por sus siglas en inglés) y el Reconocimiento de Caracteres (LPR, por sus siglas en inglés) (Laroca et al., 2025; Sonnara et al., 2025). La fase LPD constituye la primera etapa crítica y tiene como objetivo la localización espacial de la placa dentro de un fotograma mediante la predicción de cajas delimitadoras (*bounding boxes*) o polígonos de cuatro esquinas (Buleu et al., 2025; Sonnara et al., 2025; Vargoorani & Suen, 2024). La precisión en esta etapa es determinante, dado que cualquier imprecisión en las coordenadas estimadas degrada directamente el rendimiento del reconocimiento de caracteres subsecuente (Suharto et al., 2025; Swati et al., 2024; Xu et al., 2026).

La fase LPR consiste en la decodificación de la información visual contenida en la región detectada para su transformación en una cadena de texto alfanumérico (Kuyunsa et al., 2025; Wijaya & Soewito, 2024). Históricamente, esta tarea se abordaba mediante un enfoque basado en segmentación, el cual intentaba aislar cada carácter individualmente antes de clasificarlo; sin embargo, este paradigma presenta una fragilidad intrínseca ante el ruido visual, caracteres adheridos y variaciones de iluminación (Sonnara et al., 2025; Xiong et al., 2026; Zherzdev & Gruzdev, 2018).

En contraste, el estado del arte ha convergido hacia el enfoque libre de segmentación (*segmentation-free*), el cual trata la lectura de la placa como un problema de etiquetado secuencial holístico (Kuyunsa et al., 2025; Sonnara et al., 2025; Zherzdev & Gruzdev, 2018). Esta metodología emplea redes neuronales recurrentes (RNN) o mecanismos de atención (*Transformers*) junto a una función de pérdida de Clasificación Temporal Conexionista (CTC), permitiendo que el sistema aprenda a reconocer la secuencia completa de la patente sin necesidad de pre-segmentar los caracteres manualmente, lo que incrementa significativamente la robustez del prototipo en entornos no restringidos (Abdul-Sattar & Al Azzo,

2026; Kuyunsa et al., 2025; Mobeen, 2026; Zherzdev & Gruzdev, 2018).

1.1.3 Flujos de Procesamiento: *Pipelines* Secuenciales vs. Sistemas Integrados (*End-to-End*)

La eficiencia operativa de un sistema ALPR en tiempo real depende críticamente de la organización lógica de sus componentes, distinguiéndose en la literatura dos enfoques arquitectónicos predominantes: los *pipelines* secuenciales multietapa y los sistemas integrados extremo a extremo (*end-to-end*) (Sonnara et al., 2025; Xu et al., 2026).

El enfoque secuencial o multietapa es el diseño más extendido y se basa en la ejecución de modelos independientes para la localización (LPD) y el reconocimiento (LPR) (Ammar et al., 2023; Laroca et al., 2025; Mobeen, 2026). En este flujo, una primera red neuronal detecta la placa y genera un recorte de la imagen (*crop*), el cual es preprocesado y enviado a un segundo motor de OCR para la extracción de texto (Kuyunsa et al., 2025; Mobeen, 2026). Si bien este paradigma permite una mayor modularidad y facilidad de entrenamiento individual, presenta dos desventajas críticas para dispositivos de borde: la redundancia paramétrica, causada por la duplicación de extractores de características (*backbones*) en cada etapa, y la propagación de errores, donde cualquier imprecisión en la detección inicial degrada irreversiblemente la calidad del reconocimiento subsecuente (Sonnara et al., 2025; Xu et al., 2026).

Frente a estas limitaciones, han surgido los sistemas integrados extremo a extremo de un solo paso (*single-pass*), los cuales unifican la detección y el reconocimiento en un único grafo computacional (Buleu et al., 2025; Sonnara et al., 2025; Zherzdev & Gruzdev, 2018). Estas arquitecturas, como el modelo Light-Edge, emplean un *backbone* compartido (frecuentemente basado en ResNet o MobileNet) que extrae un mapa de características único para ambas tareas, eliminando transferencias de datos ineficientes entre CPU y GPU (Sonnara et al., 2025). Al optimizar una función de pérdida conjunta, estos modelos reducen la latencia de inferencia de forma drástica —logrando tasas de procesamiento superiores a los 14 FPS en hardware como NVIDIA Jetson Nano— y minimizan la huella de memoria, consolidándose como el estándar de mayor proyección cientí-

fica para el despliegue de soluciones inteligentes en el borde periférico (Sonnara et al., 2025; Zherzdev & Gruzdev, 2018).

1.2 Análisis del Estado del Arte en Detección (LPD)

La arquitectura de un sistema ALPR comienza con la capacidad de aislar la región de interés (ROI) que contiene la placa patente dentro de una escena compleja. En la literatura científica, los algoritmos de detección de objetos se dividen taxonómicamente en dos categorías: detectores de dos etapas (*two-stage*) y detectores de una sola etapa (*one-stage*) (Abdul-Sattar & Al Azzo, 2026; Xu et al., 2026). Mientras que los modelos de dos etapas (como Faster R-CNN) utilizan una Red de Propuesta de Regiones (RPN) para preseleccionar candidatos, los modelos de una sola etapa realizan la clasificación y la regresión de coordenadas en un único paso computacional. Esta distinción es crítica para el control de acceso vehicular, ya que la naturaleza estocástica del tráfico exige algoritmos que prioricen la eficiencia temporal sin sacrificar la robustez espacial (Oluwaseyi et al., 2025; Swati et al., 2024; Xu et al., 2026).

1.2.1 La Evolución de la Familia YOLO (*You Only Look Once*)

Los modelos de la serie YOLO se han consolidado como el estándar de la industria debido a su capacidad para reformular la detección como un problema de regresión unificado (Athilakshmi et al., 2023; Mobeen, 2026). Su evolución técnica refleja una búsqueda constante por optimizar el flujo de gradientes y reducir la latencia de post-procesamiento.

YOLOv5: Representa la madurez de las arquitecturas basadas en cajas de anclaje (*anchor-based*). Utiliza descriptores geométricos predefinidos que el modelo emplea como referencia para ajustar las coordenadas del objeto (Athilakshmi et al., 2023; Oluwaseyi et al., 2025). Aunque su velocidad es sobresaliente, alcanzando los 92 FPS en entornos controlados, su rigidez ante cambios de escala limita su eficacia cuando la cámara de portería captura vehículos a diferentes distancias o ángulos oblicuos (Oluwaseyi et al., 2025; Swati et al., 2024).

No obstante, su estabilidad la posiciona como un *baseline* confiable para este proyecto en caso de requerir una implementación de baja complejidad.

YOLOv7: Introdujo innovaciones en el entrenamiento mediante estrategias de “bolsa de obsequios” (*bag-of-freebies*), optimizando el costo de inferencia sin aumentar la carga paramétrica (Wang et al., 2023 citado en Mobeen, 2026; Wijaya y Soewito, 2024). Su arquitectura destaca por el uso de redes de agregación de capas eficientes, logrando un balance que la hace viable para dispositivos con recursos limitados como la Raspberry Pi, permitiendo detecciones precisas en sistemas de portones inteligentes con una latencia controlada (Wijaya & Soewito, 2024).

YOLOv8: Marcó un hito al adoptar un cabezal de detección libre de anclas (*anchor-free*), lo que permite predecir directamente el centro de los objetos en lugar de ajustar cajas predefinidas, agilizando notablemente la convergencia (Mobeen, 2026; Ultralytics, 2025a). Técnicamente, su éxito reside en el módulo C2f (Cross-Stage Partial bottleneck con dos convoluciones), el cual combina características de alto nivel con flujos de gradientes optimizados, permitiendo al modelo aprender representaciones ricas con una menor densidad paramétrica (Mobeen, 2026; Ultralytics, 2025a). Esta capacidad de representar detalles de grano fino hace de YOLOv8s un candidato ideal para el prototipo UTEM, asegurando la detección de placas bajo condiciones ruidosas.

YOLOv10: Revolucionó el despliegue en el borde mediante la implementación de la Doble Asignación Consistente durante el entrenamiento. Este mecanismo emplea simultáneamente una estrategia “uno-a-muchos” (para proporcionar señales de supervisión ricas) y una “uno-a-uno” que permite una inferencia nativa libre de Supresión de No Máximos (NMS-free) (Abdul-Sattar & Al Azzo, 2026; Ultralytics, 2025a, 2025b). Al eliminar el cuello de botella del NMS, que tradicionalmente se ejecuta en CPU, YOLOv10 logra una latencia predecible y una eficiencia superior en dispositivos embebidos, manteniendo precisiones de hasta 99.16 % en escenarios de tráfico denso (Meesad & Thumthong, 2025).

YOLOv11 y YOLOv12: Representan la integración de mecanismos de atención espacial interna. YOLOv12, lanzado en febrero de 2025, emplea arquitecturas centradas en la atención de largo alcance, lo que le permite “limpiar” el mapa de características y enfocarse en la placa incluso ante fondos complejos o

degradaciones climáticas (Buleu et al., 2025; Suharto et al., 2025). Resultados experimentales demuestran que YOLOv12 logra un incremento del 2.3 % en precisión sobre la v11, estableciendo la frontera actual de exactitud para sistemas inteligentes de transporte (Buleu et al., 2025).

YOLOv26: La iteración más reciente (enero 2026) se enfoca en el dominio de la inferencia por CPU, logrando ser un 43 % más rápida que generaciones previas. Esto se consigue mediante la eliminación de la Pérdida Focal de Distribución (DFL), simplificando la representación probabilística de las cajas delimitadoras para aliviar la carga de los procesadores de bajo costo (Ultralytics, 2025a, 2025b). Además, introduce el optimizador MuSGD, inspirado en los paradigmas de estabilidad de los grandes modelos de lenguaje, garantizando que el entrenamiento sea resiliente ante la escasez de datos reales (Ultralytics, 2025a, 2025b). Esta arquitectura representa el estándar de “futuro asegurado” para el prototipo propuesto en esta investigación.

1.2.2 Arquitecturas de Detección Alternativas

A pesar del dominio de la familia YOLO, la literatura documenta alternativas que ofrecen ventajas específicas según el entorno de despliegue:

Faster R-CNN: Es el referente de los detectores de dos etapas. Al utilizar un extractor de características (*Backbone*) como Inception V2, logra una precisión superior en la detección de patrones visuales extremadamente finos (Kuyunsa et al., 2025). Sin embargo, su naturaleza secuencial impone una latencia elevada (aprox. 4.5 ms/imagen en GPUs de alto rendimiento), lo que la hace inviable para el control de acceso en tiempo real sobre hardware de bajo costo como el que se validará en el Campus Ñuñoa (Meesad & Thumthong, 2025; Swati et al., 2024).

EfficientDet-D0: Desarrollada para maximizar la eficiencia paramétrica mediante redes de pirámide de características bidireccionales (BiFPN), permitiendo una fusión de características de diferentes escalas de forma equilibrada (Swati et al., 2024). Aunque su precisión en localización es competitiva, su rendimiento suele ser inferior a las versiones recientes de YOLO cuando se enfrenta a distorsiones de perspectiva severas en el flujo vehicular continuo (Swati et al., 2024).

SSD (Single Shot Detector): Fue pionera en la detección de un solo paso,

prediciendo cajas desde múltiples mapas de características. No obstante, estudios recientes indican que presenta fluctuaciones de precisión ante objetos pequeños (como placas distantes) y una menor capacidad de generalización comparada con los modelos *anchor-free* modernos, situándose frecuentemente al final de los *benchmarks* de robustez ambiental (Meesad & Thumthong, 2025; Swati et al., 2024).

1.3 Motores de Extracción de Texto (LPR) y OCR

Una vez aislada la región de la placa patente, el sistema debe transcribir los patrones visuales en cadenas alfanuméricas estructuradas. La eficiencia de este proceso es determinante para el rendimiento global del sistema de control de acceso.

1.3.1 Metodologías de Reconocimiento

Históricamente, el reconocimiento de caracteres vehiculares se abordó mediante el enfoque basado en segmentación. Este paradigma tradicional fragmenta la placa detectada en sub-imágenes que contienen letras y números individuales para ser procesados por clasificadores independientes (Xu et al., 2026). Arquitectónicamente, depende de algoritmos de binarización y componentes conectados para aislar los glifos, lo que lo hace extremadamente vulnerable a fallos ante caracteres adheridos, ruido visual, desenfoque por movimiento o variaciones en la alineación del vehículo (Sonnara et al., 2025; Xu et al., 2026; Zherzdev & Gruzdev, 2018). Cualquier error en la segmentación inicial resulta en una falla irreversible en la etapa de clasificación (Xiong et al., 2026).

En contraposición, las arquitecturas modernas adoptan un enfoque libre de segmentación (*segmentation-free*), tratando la lectura de la placa como un problema de etiquetado secuencial holístico. Este método procesa la imagen completa como una secuencia continua de características espaciales, evitando la frágil etapa de binarización manual y recorte (Kuyunsa et al., 2025; Xu et al., 2026). Al permitir que la red aprenda representaciones directamente de los píxeles sin procesar, se incrementa significativamente la robustez en entornos no restringidos

(Abdul-Sattar & Al Azzo, 2026; Sonnara et al., 2025).

Arquitecturas CRNN y el Rol del CTC: El estándar de facto para este enfoque holístico son las Redes Neuronales Recurrentes Convolucionales (CRNN). Estas combinan las fortalezas de las redes convolucionales (CNN) para la extracción de características con las redes recurrentes (RNN) para el modelado de secuencias (Vargoorani & Suen, 2024; Zherzdev & Gruzdev, 2018). Internamente, una CNN (como ResNet o MobileNet) actúa como *backbone* para generar un mapa de características. Luego, capas bidireccionales de Memoria a Corto y Largo Plazo (Bi-LSTM) escanean este mapa para capturar dependencias contextuales entre caracteres, entendiendo el orden y la estructura sintáctica de la patente (Kuyunsa et al., 2025; Mobeen, 2026; Vargoorani & Suen, 2024).

Para viabilizar el entrenamiento de extremo a extremo sin etiquetas segmentadas a nivel de carácter, la capa de Clasificación Temporal Conexiónista (CTC) resulta indispensable (Sonnara et al., 2025; Zherzdev & Gruzdev, 2018). El CTC resuelve el problema de alineación prediciendo una distribución de probabilidad para cada intervalo espacial, utilizando un carácter “blanco” (*blank*) para manejar espacios o separar caracteres repetidos (Mobeen, 2026; Sonnara et al., 2025; Zherzdev & Gruzdev, 2018).

Transformers y Mecanismos de Atención: La literatura más reciente documenta la evolución hacia mecanismos de atención, sustituyendo las capas LSTM por bloques tipo Transformer. Modelos como el Spatial Visual Transformer (SVTR) permiten capturar dependencias espaciales de largo alcance, resultando en mapas de características más limpios y resilientes ante ruidos o ángulos inclinados (Buleu et al., 2025; Xu et al., 2026). Incluso, se aprovecha la atención global para mejorar el reconocimiento en capturas borrosas de baja resolución (Azadbakht et al., 2022 citado en Xiong et al., 2026).

1.3.2 Benchmarking de Motores y Viabilidad en el Borde

La elección del motor OCR impacta directamente en la viabilidad técnica del despliegue en el Campus Ñuñoa, donde las restricciones de *hardware* exigen un estricto equilibrio entre latencia y precisión.

Tesseract OCR: Mantenido históricamente por Google, Tesseract utiliza

redes LSTM, pero su diseño está fuertemente optimizado para documentos impresos estáticos (Athilakshmi et al., 2023; Meesad & Thumthong, 2025; Swati et al., 2024). Su naturaleza secuencial de ejecución en CPU restringe su rendimiento a aproximadamente 25 lecturas por minuto, haciéndolo ineficiente para flujos de video continuo (Xu et al., 2026). Adicionalmente, experimenta una severa degradación de precisión ante distorsiones de perspectiva o rotaciones vehiculares (Swati et al., 2024; Xu et al., 2026).

EasyOCR: Esta biblioteca basada en PyTorch emplea un detector de texto CRAFT acoplado a una red CRNN con decodificador CTC (Kuyunsa et al., 2025; Mobeen, 2026; Xu et al., 2026). Si bien exhibe una alta robustez ante tipografías complejas y placas deterioradas, su carga computacional en entornos carentes de aceleración gráfica es crítica. Su ejecución en CPU apenas alcanza 8 lecturas por minuto, mostrando niveles de confianza altamente oscilatorios (promedio de 0.41) en secuencias de video real (Mobeen, 2026; Xu et al., 2026).

PaddleOCR: Posicionado como un pipeline de grado industrial, integra el algoritmo de binarización DBNet con el modelo de reconocimiento SVTR basado en Transformers (Buleu et al., 2025; Xu et al., 2026). Es el motor más rápido y preciso documentado, logrando procesar hasta 120 lecturas por minuto (en GPU) y alcanzando precisiones del 99.6 % al integrarse con versiones recientes de YOLO (Buleu et al., 2025; Xu et al., 2026). Su manejo eficiente de texto ruidoso y placas oblicuas lo establece como la alternativa de código abierto predominante para aplicaciones ALPR (Xu et al., 2026).

Impacto en Hardware de Borde: Para el prototipo UTEM, la NVIDIA Jetson Nano emerge como la plataforma ideal, aprovechando sus 128 núcleos CUDA para optimizar arquitecturas como PaddleOCR mediante TensorRT. La cuantización del modelo a enteros de 8 bits (INT8) garantiza tasas superiores a los 14 FPS, satisfaciendo la cuota de latencia de 100 ms exigida para sistemas de barrera automatizada (Sonnara et al., 2025; Swati et al., 2024; Xu et al., 2026). En contraste, la utilización de una Raspberry Pi convencional requeriría forzosamente la integración de un acelerador NPU dedicado (como el Hailo-8L) para evitar que el cuello de botella de inferencia por CPU impida una operación fluida (Sonnara et al., 2025; Xu et al., 2026).

1.4 Hardware de Borde y Optimización (*Edge AI*)

La transición de modelos desde servidores de alto rendimiento hacia el borde de la red (*Edge AI*) es un requerimiento fundamental para sistemas ALPR operativos, donde la dependencia de la nube introduce latencias inaceptables y riesgos de privacidad. Sin embargo, el despliegue local impone severas restricciones de memoria, consumo energético y capacidad de cómputo.

1.4.1 Ecosistemas de Cómputo en el Borde: Arquitecturas y Rendimiento

La literatura documenta dos grandes familias de ecosistemas de cómputo para el despliegue de redes neuronales profundas en el borde: aquellas con aceleración nativa por GPU y aquellas basadas primordialmente en procesamiento de CPU.

Aceleración Nativa por GPU (Arquitectura NVIDIA Jetson): La familia NVIDIA Jetson (incluyendo Nano, Xavier AGX y Orin) está diseñada específicamente para satisfacer las demandas de cómputo paralelo masivo del *Deep Learning* (Ammar et al., 2023; Sonnara et al., 2025). A diferencia de las CPUs convencionales, la Jetson Nano integra una GPU de arquitectura Maxwell con 128 núcleos CUDA, lo que permite la ejecución concurrente de operaciones tensoriales. Esta paralelización optimiza el rendimiento en modelos de detección de una sola etapa como YOLOv8 y YOLOv10 (Meesad & Thumthong, 2025; Sonnara et al., 2025). El ecosistema se apoya en la suite NVIDIA JetPack, la cual incluye el motor de inferencia TensorRT, capaz de optimizar los grafos computacionales para maximizar la tasa de procesamiento (FPS) y cumplir con el presupuesto de latencia de ≈ 100 ms, métrica exigida para el control de acceso vehicular en tiempo real (Ammar et al., 2023; Sonnara et al., 2025).

Procesamiento Basado en CPU (Arquitectura Raspberry Pi): Por otro lado, plataformas como la Raspberry Pi 5 utilizan procesadores de propósito general (e.g., Broadcom BCM2712 Cortex-A76) que, si bien son eficientes en tareas lógicas secuenciales, carecen de unidades de procesamiento gráfico compatibles con marcos de IA de alto nivel como CUDA (Oluwaseyi et al., 2025). Ejecutar modelos de detección complejos directamente en su CPU resulta en una latencia

prohibitiva para aplicaciones vehiculares, logrando frecuentemente tasas de inferencia inferiores a 5 FPS (Oluwaseyi et al., 2025; Xu et al., 2026). Esto podría traducirse en la pérdida de detección de vehículos con movimiento rápido en entornos de portería.

Para viabilizar el uso de placas como la Raspberry Pi en tareas de inferencia pesada, resulta imperativa la incorporación de Unidades de Procesamiento Neuronal (NPU) externas, como el módulo AI HAT+ basado en chips Hailo-8L (Oluwaseyi et al., 2025). Arquitectónicamente, estas NPUs están optimizadas para los flujos de datos de redes neuronales, ejecutando cálculos matriciales con una eficiencia energética (TOPS por Watt) muy superior a la de una CPU convencional (Oluwaseyi et al., 2025; Xu et al., 2026).

1.4.2 Técnicas de Compilación y Cuantización

Fundamentos de la Cuantización: Para adaptar modelos masivos a las restricciones de memoria del hardware de borde, se emplea el proceso de cuantización. Esta técnica consiste en reducir la precisión numérica de los pesos y las funciones de activación de la red neuronal, trasladándolos desde representaciones de punto flotante de 32 bits (FP32) a formatos más compactos como 16 bits (FP16) o números enteros de 8 bits (INT8) (Sonnara et al., 2025; Swati et al., 2024). Matemáticamente, esto implica mapear los valores continuos del modelo original a un conjunto discreto de niveles, utilizando frecuentemente la calibración por divergencia de Kullback-Leibler (KL) para minimizar la distorsión estadística de los datos (Ammar et al., 2023).

Beneficios en Memoria y Aceleración de Inferencia: La transición de FP32 a INT8 reduce la huella del modelo en disco y RAM en un factor de hasta 4x. En contextos operativos, modelos ALPR con un peso inicial de 37 MB logran comprimirse a 11 MB al ser cuantizados a 16 bits, un factor crítico para microordenadores con recursos limitados (e.g., los 4 GB de RAM de la Jetson Nano) (Sonnara et al., 2025; Swati et al., 2024). A nivel de ejecución, el menor tamaño de los datos disminuye drásticamente el ancho de banda requerido para las transferencias de memoria. Además, el hardware moderno incorpora instrucciones SIMD que permiten procesar múltiples operaciones INT8 en un único ciclo de

reloj, posibilitando aceleraciones de inferencia de hasta 4.6x (Ammar et al., 2023; Sonnara et al., 2025).

Rol de Motores de Optimización: La eficiencia en el despliegue requiere de compiladores especializados. TensorRT, por ejemplo, ejecuta una "fusión de capas" (*Layer & Tensor Fusion*), combinando operaciones de convolución, sesgo y activación en un único *kernel* de cómputo, lo que mitiga las ineficientes transferencias de datos en memoria (Ammar et al., 2023; Sonnara et al., 2025). Por su parte, TensorFlow Lite ofrece formatos de serialización (*FlatBuffers*) e intérpretes optimizados para arquitecturas ARM, facilitando el despliegue en microprocesadores móviles (Swati et al., 2024).

Impacto en la Precisión (mAP): Pese a la compresión numérica, la literatura empírica demuestra que la pérdida de precisión media (mAP) es marginal cuando se emplean técnicas de calibración rigurosas. En evaluaciones del modelo Light-Edge optimizado mediante Torch-TensorRT a INT8, se reportó un incremento de velocidad de 3.1 FPS a 14.2 FPS, incurriendo en una penalización de apenas 0.4 puntos porcentuales en mAP respecto al modelo FP32 base (Sonnara et al., 2025). Esto evidencia que la cuantización no es una concesión comprometedora, sino la técnica habilitadora que hace factible el funcionamiento de arquitecturas de *Deep Learning* en escenarios comerciales de borde (Sonnara et al., 2025; Swati et al., 2024).

1.5 Robustez en Entornos No Restringidos

1.5.1 Técnicas de Preprocesamiento Crítico

1.5.2 Módulos de Mejora Dinámica

1.5.3 Datos Sintéticos y Aumento (*Data Augmentation*)

1.6 Consistencia Temporal y Seguimiento (*Tracking*)

1.6.1 Algoritmos de *Tracking*

1.6.2 Votación Temporal e Interpolación

1.7 Contexto Normativo y Operacional Chileno

1.7.1 Estándares de Patentes en Chile

1.7.2 Requerimientos Funcionales del Prototipo

CAPÍTULO 2

METODOLOGÍA

2.1 Diseño metodológico

[Describe el tipo de investigación o metodología empleada en tu proyecto (por ejemplo, desarrollo ágil de software, investigación experimental, diseño cuantitativo o cualitativo, etc.) y justifica su elección.]

2.2 Fases o etapas del proyecto

[Detalla las diferentes fases del desarrollo de tu proyecto. Si sigues una metodología de desarrollo de software como Scrum, XP o Cascada, describe cada una de sus fases adaptadas a tu tesis.]

2.2.1 Etapa 1: Levantamiento de requerimientos y análisis

[Describe cómo se identificaron las necesidades y requerimientos iniciales del sistema o de la investigación.]

2.2.2 Etapa 2: Diseño y arquitectura

[Explica la fase de diseño del sistema, modelado de base de datos, arquitectura de software, diagramas UML, etc.]

2.2.3 Etapa 3: Desarrollo e implementación

[Menciona brevemente cómo se llevó a cabo la construcción o programación del sistema/solución.]

2.2.4 Etapa 4: Pruebas y validación

[Describe cómo se validaron los resultados o el correcto funcionamiento del sistema desarrollado.]

2.3 Herramientas y tecnologías

[Describe las herramientas, lenguajes de programación, frameworks, plataformas y bases de datos utilizadas en tu proyecto, y por qué fueron elegidos.]

2.4 Sujetos o elementos de estudio (Población y Muestra)

[Si aplica, define sobre qué universo, muestra de usuarios, datos o infraestructura se aplicó el estudio o se validó la solución.]

CAPÍTULO 3

DESARROLLO DE LA SOLUCIÓN

3.1 Arquitectura de la solución

[Presenta aquí el esquema general de tu sistema (patrón arquitectónico como MVC, microservicios, cliente-servidor, etc.). Puedes incluir diagramas de arquitectura en la carpeta 'figuras/' e insertarlos con el comando `\includegraphics.`]

3.2 Modelo de datos y almacenamiento

[Detalla el diseño de tu base de datos o estructura de almacenamiento. Incluye diagramas entidad-relación (DER) o esquemas NoSQL y explica las tablas o colecciones principales.]

3.3 Implementación detallada de componentes

[Describe la programación y construcción de los módulos principales del sistema, incluyendo algoritmos clave, consumo de APIs, servicios en la nube o lógica del negocio relevante.]

3.3.1 Componente o módulo backend

[Descripción técnica del desarrollo del lado del servidor, endpoints principales, etc.]

3.3.2 Componente o módulo frontend (Interfaz de usuario)

[Descripción técnica del desarrollo del lado del cliente, diseño de interfaces, etc.]

3.4 Casos de uso principales y flujos de trabajo

[Explica cómo los usuarios interactúan con el sistema mediante flujos clave. Apóyate de diagramas de secuencia o capturas de pantalla de la interfaz para ilustrar los procesos principales.]

CAPÍTULO 4

RESULTADOS Y DISCUSIÓN

4.1 Plan de pruebas y validación

[Describe la estrategia utilizada para verificar que el sistema funciona correctamente y cumple con los requerimientos técnicos y operacionales.]

4.1.1 Pruebas unitarias y de integración

[Describe cómo se probaron los componentes individuales y su interacción conjunta.]

4.1.2 Pruebas de rendimiento o estrés

[Si aplica, detalla los tiempos de respuesta del sistema bajo carga, consumo de recursos, etc.]

4.1.3 Pruebas de aceptación de usuarios

[Describe si se realizaron pruebas con usuarios reales (encuestas de usabilidad, métricas de satisfacción, System Usability Scale - SUS, etc.).]

4.2 Presentación de resultados

[Muestra los datos obtenidos de las pruebas mediante tablas, gráficos o capturas. Asegúrate de referenciar todas tus figuras y tablas en el texto (ejemplo: “como se observa en la Tabla 4.1”).]

Tabla 4.1. Ejemplo de tabla de resultados

Métrica de Evaluación	Valor Esperado	Valor Obtenido
Tiempo de carga de página	<2.0 s	1.45 s
Precisión del algoritmo	>90 %	93.2 %
Tasa de éxito de transacciones	100 %	100 %

4.3 Discusión de los resultados

[Analiza críticamente los resultados obtenidos. Compara tu propuesta frente a soluciones existentes (estado del arte) y explica cómo se resolvieron las limitaciones del problema planteado.]

CONCLUSIONES

Cumplimiento de Objetivos

[Evalúa críticamente cómo se alcanzaron los objetivos propuestos (tanto el general como los específicos) y qué resultados concretos respaldan este cumplimiento.]

Aportes y Contribuciones

[Describe el valor agregado de tu trabajo. ¿Qué aporta a la disciplina, a la institución, a la empresa (si aplica) o al estado del arte?]

Limitaciones de la Solución

[Describe con honestidad técnica las restricciones y limitaciones de tu desarrollo o de tu investigación (por ejemplo, dependencias tecnológicas, falta de datos, limitaciones de rendimiento, etc.).]

Trabajos Futuros

[Propón sugerencias, recomendaciones o nuevas líneas de trabajo, mejoras de software y extensiones de la investigación que quedaron fuera del alcance de este proyecto pero que son viables para continuarse en el futuro.]

BIBLIOGRAFÍA

- Abdul-Sattar, N. M., & Al Azzo, F. (2026). (PDF) Real-Time License Plate Recognition Using YOLOv10 and OCR Integration for Autonomous Traffic Monitoring and Surveillance Systems. *ResearchGate*. <https://doi.org/10.56286/vbsyx52>
- Ammar, A., Koubaa, A., Boulila, W., Benjdira, B., & Alhabashi, Y. (2023). A Multi-Stage Deep-Learning-Based Vehicle and License Plate Recognition System with Real-Time Edge Inference. *Sensors*, 23(4), 2120. <https://doi.org/10.3390/s23042120>
- Athilakshmi, R., Khatoria, R., & Srinivasan, S. (2023). Automatic Number Plate Detection Using Deep Learning Techniques. *Recent Trends in Data Science and Its Applications*, 130-135. <https://doi.org/10.13052/rp-9788770040723.026>
- Buleu, B., Robu, R., & Filip, I. (2025). A Deep Learning-Based System for Automatic License Plate Recognition Using YOLOv12 and PaddleOCR. *Applied Sciences*, 15(14), 7833. <https://doi.org/10.3390/app15147833>
- Kuyunsa, A. M., Mbayandjambe, A. M., Awoopeur, V., Nkwimi, G. B., Byaombe, D. M., Kutuka, X. F., Nguemdjom, D. K. T., & Kandolo, H. (2025). Deep Learning-Enhanced Automatic License Plate Recognition: A CNN-Based Framework for Real-Time Traffic Management Systems. *Revue Internationale de la Recherche Scientifique (Revue-IRS)*, 3(3), 2930-2942. <https://doi.org/10.5281/zenodo.15657392>
- Laroca, R., Estevam, V., Moreira, G. J. P., Minetto, R., & Menotti, D. (2025). Advancing Multinational License Plate Recognition Through Synthetic and Real Data Fusion: A Comprehensive Evaluation [Comment: IET Intelligent Transport Systems, vol. 19, no. 1, p. e70086, 2025]. *IET Intelligent Transport Systems*, 19(1), e70086. <https://doi.org/10.1049/itr2.70086>
- Meesad, P., & Thumthong, W. (2025). Advanced Deep Learning Techniques for Automated License Plate Recognition. *Scientific Reports*, 15, 41194. <https://doi.org/10.1038/s41598-025-24967-9>
- Mobeen, M. M. (2026, junio). Real-Time Automatic License Plate Recognition Using YOLOv8, SORT Tracking, and Temporal Data Interpolation [Com-

- ment: 7 Pages, For Accessing code:<https://github.com/mobeen-pmo/Automatic-License-Plate-Recognition>]. <https://doi.org/10.48550/arXiv.2606.04684>
- Oluwaseyi, O., Iheagwara, S., Lakan, N., & Akanbi, A. (2025). A Comparative Analysis of YOLOv5, YOLOv8, and YOLOv10 for Balancing Accuracy and Speed for Real-Time Vehicle Detection. *Nature Journal of Emerging Sciences Technologies and Innovations*, 3, 117-130. <https://doi.org/10.65752/wb2sep51>
- Sonnara, F., Chihaoui, H., & Filali, F. (2025). Efficient Real-Time License Plate Recognition Using Deep Learning on Edge Devices. *Journal of Real-Time Image Processing*, 22. <https://doi.org/10.1007/s11554-025-01738-3>
- Suharto, R. N., Zajuli Al Faroby, M. H., & Setiawan, Y. (2025). Comparative Evaluation of YOLO-based Models for Indonesian License Plate Detection and Recognition with EASY OCR. *Procedia Computer Science*, 269, 943-952. <https://doi.org/10.1016/j.procs.2025.09.037>
- Swati, S., Kawa, S. D., Kamble, S., Desai, D., Karelia, P. H., & Engineer, P. (2024). An Efficient Deep Learning Based License Plate Recognition for Smart Cities. *ELCVIA Electronic Letters on Computer Vision and Image Analysis*, 23(2), 50-64. <https://doi.org/10.5565/rev/elcvia.1917>
- Ultralytics. (2025a, enero). YOLOv10 vs YOLOv8 Comparison.
- Ultralytics. (2025b, enero). YOLOv8 vs YOLOv10 Comparison.
- Vargoorani, Z. E., & Suen, C. Y. (2024). License Plate Detection and Character Recognition Using Deep Learning and Font Evaluation [Comment: 12 pages, 5 figures. This is the pre-Springer final accepted version. The final version is published in Springer, Lecture Notes in Computer Science (LNCS), Volume 14731, 2024. Springer Version of Record]. https://doi.org/10.1007/978-3-031-71602-7_20
- Wijaya, V. F., & Soewito, B. (2024). Efficient License Plate Detection and Recognition with YOLOv7 and OCR. *International Journal of Intelligent Systems and Applications in Engineering*, 12(3), 1598-1605.
- Xiong, W., Cao, L., Yan, D., Jiang, Y., Zhang, G., Wang, Y., Huang, X., & Liu, L. (2026). License Plate Recognition Methodology in Complex Scenarios Based on CSCM-YOLOv8 and CSM-LPRNet. *PLOS One*, 21(1), e0339649. <https://doi.org/10.1371/journal.pone.0339649>

- Xu, L., Shang, Z., Chen, X., Zeng, T., Dai, W., & Zhao, J. (2026). Deep Learning Algorithms for License Plate Recognition: A Review. *Neural Networks*, 200, 108842. <https://doi.org/10.1016/j.neunet.2026.108842>
- Zherzdev, S., & Gruzdev, A. (2018, junio). LPRNet: License Plate Recognition via Deep Neural Networks. <https://doi.org/10.48550/arXiv.1806.10447>

GLOSARIO

API	(<i>Application Programming Interface</i>): Interfaz de programación de aplicaciones. Conjunto de definiciones y protocolos para el desarrollo e integración de software.
CSS	(<i>Cascading Style Sheets</i>): Hojas de estilo en cascada, utilizadas para definir la presentación visual de documentos estructurados en HTML o XML.
HTML	(<i>HyperText Markup Language</i>): Lenguaje de marcado de hipertexto. Es el estándar para la estructura y contenido de las páginas web.
LaTeX	Sistema de preparación de documentos para la composición tipográfica de alta calidad, ampliamente utilizado en textos científicos y académicos.
REST	(<i>Representational State Transfer</i>): Transferencia de estado representacional. Estilo arquitectónico para el diseño de servicios web.
UML	(<i>Unified Modeling Language</i>): Lenguaje de modelado unificado. Estándar visual para especificar, construir y documentar sistemas de software.
UTEM	Universidad Tecnológica Metropolitana.

ANEXOS

ANEXO A

MANUAL DE INSTALACIÓN Y CONFIGURACIÓN

[Describe en este anexo el paso a paso para instalar, configurar y ejecutar tu solución tecnológica...]

A.1 Requisitos del sistema

[Especifica los requisitos mínimos y recomendados de hardware, sistema operativo, bases de datos y entornos de ejecución.]

A.2 Instrucciones de instalación y despliegue

[Describe detalladamente los comandos, configuraciones de variables de entorno y pasos necesarios para desplegar la solución.]

ANEXO B

CÓDIGO FUENTE Y CONFIGURACIÓN RELEVANTE

[Puedes incluir listados de scripts de base de datos (SQL, NoSQL), archivos de configuración (Docker, variables de entorno, webpack, etc.) o fragmentos de código fuente que complementen el cuerpo de la obra.]